

PROJECT DOCUMENTATION: THE 5x5 BOMB TIC-TAC-TOE

1. Project Introduction

I created a terminal-based 5x5 Tic-Tac-Toe game using **Python**. The game is designed from scratch using only foundational programming concepts suitable for beginners.

Game Rules:

- **The Grid:** A 5x5 board layout. To win, 4 consecutive matching marks are required (horizontal, vertical, or diagonal).
- **Players:** Player O (Me) and Player X (The Computer).
- **Hidden Bomb:** A random square holds a secret bomb. Stepping on it completely wipes out that turn.
- **Removal Privilege:** Each side can completely erase one opponent piece, once per game.

2. File Directory

My repository contains the following files pushed directly to **GitHub**:

text

tic-tac-toe-bomb/

|

|— tic_tac_toe_plus.ipynb # The game file

|— README.md # This project documentation file

3. My Debugging Log: Mistakes I Made and Fixed

While building and compiling the game from a Jupyter Notebook file (.ipynb) into a clean Python script (.py), I encountered several syntax, loop output, and indentation hurdles. Here is exactly how I diagnosed and fixed each issue step-by-step:

Issue 1: Game Loop Flooding the Screen

- **The Problem:** When I ran my initial script, every single turn printed a brand-new grid downwards. It created an endlessly scrolling screen that was messy to read.

- **How I Fixed It:** I realised that running loops inside Jupyter cells stacks outputs indefinitely. I fixed this by importing from `IPython.display` import `clear_output` and placing `clear_output(wait=True)` at the top of my while loop to clean the frame before redrawing the grid.

Issue 2: Variable Scope Error (`NameError: game_is_running is not defined`)

- **The Problem:** Python stopped executing and complained that `game_is_running` was not defined.
- **How I Fixed It:** I reviewed my code and found that I placed the loop condition `while game_is_running:` *before* actually defining the variable state. I reordered my variable declarations, placing `game_is_running = True` higher up in the setup phase.

Issue 3: Line 62 Alignment Blunder (`IndentationError`)

- **The Problem:** The parser threw an error pointing directly to the up-right diagonal win check statement:

text

File "<string>", line 62

```
if row_i >= 3 and col_i <= 1:
```

`IndentationError: unindent does not match any outer indentation level`

- **How I Fixed It:** Python relies strictly on matching spaces. When editing my matrix bounds, I had mixed up tab indents and blank spaces on line 62. I went to the line, deleted all front spacing, and used exact indentation block spaces to line it up perfectly with the other directional win checks.

Issue 4: Line 74 Winning Text Spill-Over (`Indentation Error`)

- **The Problem:** Right after fixing line 62, a secondary error immediately flagged line 74:

text

File "<string>", line 74

```
print("\n PLAYER", current_player, "WINS THIS ROUND!")
```

`IndentationError: unindent does not match any outer indentation level`

- **How I Fixed It:** Because line 62 was broken earlier, the nested code block below it had shifted out of position. The execution print statement was sitting floating outside the parent if has_winner: wrapper context. I resolved this by sliding the entire execution block rightward by 4 spaces so that the action statements properly belong inside the win evaluation criteria block.
-

4. Git & GitHub Publishing Logs

To track my progress and host my final working build, I successfully integrated my local folder workspace using standard terminal tracking commands:

1. **Staged all components collectively:**

```
bash
```

```
git add .
```

(This added both tic_tac_toe_plus.ipynb and its temporary recovery checkpoints into safe tracking).

2. **Committed the codebase with notes:**

```
bash
```

```
git commit -m "Fix layout alignment bugs and add clean screen delays"
```

3. **Pushed cleanly online to my main cloud stream:**

```
bash
```

```
git push origin main
```